

Team 6: Kernel Crashers

Ushma Dhakal, Evan Long, Carolina Olaya

Block Sorting Robot

Setup:

Robot:

- 2 DC motors
- Arduino Nano
- HC-05 wireless serial connection
- LS298N motor driver
- MG90S servo
- 4x AAA batteries
- Printed frame, manipulator

Environment:

- iPhone Camera (using cable or app like DroidCam to analyze photo/video feed)
- Wood frame to support the camera and keep the placement consistent.
 - possibly make short walls for our environment too
- Taped “deposit areas” to drop blocks
- Different colored blocks, a few of each color
- Calibration checkerboard, to line up our camera before each run

Operations:

Overall:

(Pre-check):

- Goals are set (we know how many blocks we want in which areas)
- Frame, camera, and static environment are set up.

Actual high-level Operation:

1. Camera analyzes environment with the calibration checkerboard down, to calibrate
2. Checkerboard is removed, blocks are placed willy-nilly
 - a. Note checkerboard, block, and robot sizes are well known and defined
3. Camera captures environment to determine where blocks and “deposit areas” are
4. Program recognizes objects, and develops a virtual environment
5. Program motion plans around the objects in the environment.

6. Program extrapolates from planned path to generate a series of commands for the robot to follow the path
7. Program begins sending our 'one letter' commands to the Arduino
8. Periodic (2x per second) analysis of the environment, comparing robot placement to planned path. If it deviates by more than an accepted error margin, repeat from step 4.
9. Repeat from step 4 to bring block to deposit area
10. Repeat from step 4 for as many blocks as determined in the goal-setting
11. Repeat steps 4-8 to place the robot in its 'home'

Arduino Control:

- Simple Programs only, command sent from the pc to the robot on how to move every 0.1 second.
 - 'F', 'B', 'S', 'R', 'L', 'G', 'D'; forward, backward, stop, right, left, grab, drop. should be all we need.
 - All motion programs (besides S) will have a 'rolling end' so motion is not jerky.
- To be clear, no sensors of any kind on the robot itself, all decisions are made through the camera above.

Packages Required:

- Python Motion Planning (possible): to plan the course of the robot
- OpenCV: Use computer vision to analyze the environment.
 - Would use a whole multitude of the functionalities here:
 - `calibrateCamera`, `findChessboardCorners` (really specific and helpful), `undistort`, `solvePnP`, `findContours`, and `matchShapes` to name a few
- PySerial: output from our computer right into the arduino live.

Other Possible Options

Robot Plant Watering Device:

- Use computer vision to determine when a plant needs water based off the angling, curvature, and color of its leaves.
- Pour exact amounts of water on a schedule.

Attendance Taking Robot:

- More motion planning to determine paths around a predefined room while making stops at every station a student would be at.
- Compares every face seen at each station to a known database (computer vision again) to determine which students are present (OpenCV), (pandas possibly)

- Tallies up how many, and which students are present, then performs one last scan of the room to compare the number of students to the number of students recorded.
(specifically cascadeclassifier)